

Project 4

Small C compiler for LLVM IR

The objective of our project this semester is to develop a prototype C compiler. While traditional compilers translate C programs for architectures such as x86, MIPS, or ARM, or into pseudo-assembly code. Our implementation will focus on generating **LLVM IR** (<https://llvm.org/docs/LangRef.html>) as the target code. **The project is divided into several parts including lexical analyzer, parser, static checker, and code generator.** In the project 4, you will need to choose proper language features which you want to develop in this course, and complete the whole compiler.

In your hand-in report, you need to have the following:

- **Define the subset of the language which you want to choose from the C subset.**
- Give a set of testing programs and the generated LLVM-IR files, which can illustrate the features of your testing programs. (at least 3 test programs)
- Use the “**ANTLR**” to help you develop your compiler.
- You can use **C**, **C++**, or **Java** to write your compiler. ([Java is recommended](#))
- Your C compiler should at least contain the following features: (基本要求)
 - Data types: `int`、`float`、`void`
 - Statements for arithmetic computation: `+`、`-`、`*`、`/`. (e.g., `a=b+2*(100-1);`)
 - Comparison expression. (e.g., `a > b`) , comparison operation: `>`、`>=`、`<`、`<=`、`==`、`!=`
 - `if-then` / `if-then-else` program constructs.
 - `printf()` function with one/two parameters. (support types: `%d`、`%f`)For example:
 - `printf("Number is %d\n", var);`
 - `printf("Hello\n");`
 - `scanf()` function. (support types: `%d`、`%f`)
 - New operation: `##`
 - `a ## b` 表示 $a^b + b^a$, `a` 與 `b` 需要是 `float` data type , 其結果也是 `float` data type , 例如: $1.5^{2.5} + 2.5^{1.5}$
 - 運算的優先權與 `*`、`/` 相同
 - The operation `##` is supported by runtime library you provided.

可擴充的其他功能:

- **Implicit Type Conversion**
 - `<integer type> op <float type>`: The integer is converted to a floating-point value before the `op` happens. The result of the operation is always a floating-point value.
 - When you assign a floating-point value to an integer variable, the compiler performs an implicit conversion that results in truncation.

- When you assign a integer value to a floating-point variable, the compiler performs an implicit conversion.
- Explicit type casting
- Nested if construct
- While-loop construct
- Nested while-loop construct
- While-loop construct + if construct
- Function call
- Simple code optimizations
- ...

完成基本要求，可獲得基本分數 80 分；若有進一步實作擴充的功能，則視功能的實作難易度，酌量加分。若有實作基本要求之外的部分，應該在 **readme file** 中詳述，以利作業批改。

Please turn in the following files to eCourse2:

- A file describes your C subset (PDF file).
- The source codes:
 - Your compiler
 - (a) ANTLR grammar file: `myCompiler.g` or `myCompiler.g4`.
 - (b) A program to call your parser: `myCompiler_test.java`.
 - (c) Runtime: `myRuntime.c`
 - Testing programs (.c) and the corresponding LLVM IR codes (.ll) generated by your compiler. (at least 3 programs)
- A readme file (**PDF file**) describes **the features of your compiler** and explains how to compile and execute your compiler.
- Makefile
- Any other required materials

請將上述的檔案打包壓縮存成 **tar.xz** 之格式，檔名為 `prj4_<學號>.tar.xz` 檔 (例如: `prj4_401234567.tar.xz`，上傳到 eCourse2。

Due Date: June 14 (Sunday), 24:00, 2026 (無法補交)